

Unit VI: Malicious Software and Intrusion

Contents:

- 6.1 Malicious software
- 6.2 Virus and its phases, Virus classification
- 6.3 Worm, Worm propagation Model, State of Worm Technology
- 6.4 Trojan Horse
- 6.5 Intrusion and Intruders
- 6.6 Intrusion Detection System
- 6.7 Analysis Approaches: Anomaly Based, Signature Based
- 6.8 Honey pots

6.1 Malicious software

Malicious software, or malware, arguably constitutes one of the most significant categories of threats to computer systems. NIST SP 800-83 (Guide to Malware Incident Prevention and Handling for Desktops and Laptops, July 2013) defines malware as “a program that is inserted into a system, usually covertly, with the intent of compromising the confidentiality, integrity, or availability of the victim’s data, applications, or operating system or otherwise annoying or disrupting the victim.” Hence, we are concerned with the threat malware poses to application programs, to utility programs such as editors and compilers, and to kernel-level programs. We are also concerned with its use on compromised or malicious websites and servers, or in especially crafted spam e-mails or other messages, which aim to trick users into revealing sensitive personal information.

Types of Malicious software (malware)

Table: Terminology for Malware

Name	Description
Advanced Persistent Threat (APT)	Cybercrime directed at business and political targets, using a wide variety of intrusion technologies and malware, applied persistently and effectively to specific targets over an extended period, often attributed to state-sponsored organizations.
Adware	Advertising that is integrated into software. It can result in pop-up ads or redirection of a browser to a commercial site.
Attack kit	Set of tools for generating new malware automatically using a variety of supplied propagation and payload mechanisms.
Auto-rooter	Malicious hacker tools used to break into new machines remotely.
Backdoor (trapdoor)	Any mechanism that bypasses a normal security check; it may allow unauthorized access to functionality in a program, or onto a compromised system.
Downloaders	Code that installs other items on a machine that is under attack. It is normally included in the malware code first inserted on to a compromised system to then import a larger malware package.
Drive-by-download	An attack using code on a compromised website that exploits a browser vulnerability to attack a client system when the site is viewed.
Exploits	Code specific to a single vulnerability or set of vulnerabilities.

Flooders (DoS client)	Used to generate a large volume of data to attack networked computer systems, by carrying out some form of denial-of-service (DoS) attack.
Keyloggers	Captures keystrokes on a compromised system.
Logic bomb	Code inserted into malware by an intruder. A logic bomb lies dormant until a predefined condition is met; the code then triggers some payload.
Macro virus	A type of virus that uses macro or scripting code, typically embedded in a document or document template, and triggered when the document is viewed or edited, to run and replicate itself into other such documents.
Mobile code	Software (e.g., script and macro) that can be shipped unchanged to a heterogeneous collection of platforms and execute with identical semantics.
Rootkit	Set of hacker tools used after attacker has broken into a computer system and gained root level access.
Spammer programs	Used to send large volumes of unwanted e-mail.
Spyware	Software that collects information from a computer and transmits it to another system by monitoring keystrokes, screen data, and/or network traffic; or by scanning files on the system for sensitive information.
Trojan horse	A computer program that appears to have a useful function, but also has a hidden and potentially malicious function that evades security mechanisms, sometimes by exploiting legitimate authorizations of a system entity that invokes it.
Virus	Malware that, when executed, tries to replicate itself into other executable machine or script code; when it succeeds, the code is said to be infected. When the infected code is executed, the virus also executes.
Worm	A computer program that can run independently and can propagate a complete working version of itself onto other hosts on a network, by exploiting software vulnerabilities in the target system, or using captured authorization credentials.
Zombie, bot	Program installed on an infected machine that is activated to launch attacks on other machines.

Advanced Persistent Threats (APTs)

Advanced Persistent Threats (APTs) have risen to prominence in recent years. These are not a new type of malware, but rather the well-resourced, persistent application of a wide variety of intrusion technologies and malware to selected targets, usually business or political. APTs are typically attributed to state-sponsored organizations, with some attacks likely from criminal enterprises as well.

APTs differ from other types of attack by their careful target selection, and persistent, often stealthy, intrusion efforts over extended periods. A number of high-profile attacks, including Aurora, RSA, APT1, and Stuxnet, are often cited as examples. They are named as a result of these characteristics:

- **Advanced:** Use by the attackers of a wide variety of intrusion technologies and malware, including the development of custom malware if required. The individual components may not necessarily be technically advanced, but are carefully selected to suit the chosen target.
- **Persistent:** Determined application of the attacks over an extended period against the chosen target in order to maximize the chance of success. A variety of attacks may be progressively, and often stealthily, applied until the target is compromised.

- **Threats:** Threats to the selected targets as a result of the organized, capable, and well-funded attackers intent to compromise the specifically chosen targets. The active involvement of people in the process greatly raises the threat level from that due to automated attacks tools, and also the likelihood of successful attack.

The aim of these attacks varies from theft of intellectual property or security- and infrastructure related data to the physical disruption of infrastructure. Techniques used include social engineering, spear-phishing e-mails, and drive-by-downloads from selected compromised Web sites likely to be visited by personnel in the target organization. The intent is to infect the target with sophisticated malware with multiple propagation mechanisms and payloads. Once they have gained initial access to systems in the target organization, a further range of attack tools are used to maintain and extend their access.

As a result, these attacks are much harder to defend against due to this specific targeting and persistence. It requires a combination of technical countermeasures. Even with current best-practice countermeasures, the use of zero-day exploits and new attack approaches means that some of these attacks are likely to succeed. Thus multiple layers of defense are needed, with mechanisms to detect, respond, and mitigate such attacks. These may include monitoring for malware command and control traffic, and detection of exfiltration traffic.

6.2 Virus and its phases, Virus classification

The first category of malware propagation concerns parasitic software fragments that attach themselves to some existing executable content. The fragment may be machine code that infects some existing application, utility, or system program, or even the code used to boot a computer system. Computer virus infections formed the majority of malware seen in the early personal computer era. The term “computer virus” is still often used to refer to malware in general, rather than just computer viruses specifically. More recently, the virus software fragment has been some form of scripting code, typically used to support active content within data files such as Microsoft Word documents, Excel spreadsheets, or Adobe PDF documents.

The Nature of Viruses

A computer virus is a piece of software that can “infect” other programs, or indeed any type of executable content, by modifying them. The modification includes injecting the original code with a routine to make copies of the virus code, which can then go on to infect other content. Computer viruses first appeared in the early 1980s, and the term itself is attributed to Fred Cohen. Cohen is the author of a groundbreaking book on the subject. The Brain virus, first seen in 1986, was one of the first to target MSDOS systems, and resulted in a significant number of infections for this time.

Biological viruses are tiny scraps of genetic code—DNA or RNA—that can take over the machinery of a living cell and trick it into making thousands of flawless replicas of the original virus. Like its biological counterpart, a computer virus carries in its instructional code the recipe for making perfect copies of itself. The typical virus becomes embedded in a program, or carrier of executable content, on a computer. Then, whenever the infected computer comes into contact with an uninfected piece of code, a fresh copy of the virus passes into the new location. Thus, the infection can spread from computer to computer, aided by unsuspecting users, who exchange these programs or carrier files on disk or USB stick; or who send them to one another over a network. In a network environment, the ability to access documents, applications, and system services on other computers provides a perfect culture for the spread of such viral code.

A virus that attaches to an executable program can do anything that the program is permitted to do. It executes secretly when the host program is run. Once the virus code is executing, it can perform any function, such as erasing files and programs that is allowed by the privileges of the current user. One reason viruses dominated the malware scene in earlier years was the lack of user authentication and access controls on personal computer systems at that time. This enabled a virus to infect any executable content on the system. Currently, a viral mode of infection is typically one of several propagation mechanisms used by contemporary malware, which may also include worm and Trojan capabilities.

A computer virus has three parts. More generally, many contemporary types of malware also include one or more variants of each of these components:

- **Infection mechanism:** The means by which a virus spreads or propagates, enabling it to replicate. The mechanism is also referred to as the infection vector.
- **Trigger:** The event or condition that determines when the payload is activated or delivered, sometimes known as a logic bomb.
- **Payload:** What the virus does, besides spreading. The payload may involve damage or may involve benign but noticeable activity.

During its lifetime, a typical virus goes through the following four phases:

- **Dormant phase:** The virus is idle. The virus will eventually be activated by some event, such as a date, the presence of another program or file, or the capacity of the disk exceeding some limit. Not all viruses have this stage.
- **Propagation phase:** The virus places a copy of itself into other programs or into certain system areas on the disk. The copy may not be identical to the propagating version; viruses often morph to evade detection. Each infected program will now contain a clone of the virus, which will itself enter a propagation phase.
- **Triggering phase:** The virus is activated to perform the function for which it was intended. As with the dormant phase, the triggering phase can be caused by a variety of system events, including a count of the number of times that this copy of the virus has made copies of itself.
- **Execution phase:** The function is performed. The function may be harmless, such as a message on the screen, or damaging, such as the destruction of programs and data files.

Most viruses that infect executable program files carry out their work in a manner that is specific to a particular operating system and, in some cases, specific to a particular hardware platform. Thus, they are designed to take advantage of the details and weaknesses of particular systems. Macro viruses however target specific document types, which are often supported on a variety of systems.

Once a virus has gained entry to a system by infecting a single program, it is in a position to potentially infect some or all of the other files on that system with executable content when the infected program executes, depending on the access permissions the infected program has. Thus, viral infection can be completely prevented by blocking the virus from gaining entry in the first place. Unfortunately, prevention is extraordinarily difficult because a virus can be part of any program outside a system. Thus, unless one is content to take an absolutely bare piece of iron and write all one's own system and application programs, one is vulnerable. Many forms of infection can also be blocked by denying normal users the right to modify programs on the system.

Macro and Scripting Viruses

In the mid-1990s, macro or scripting code viruses became by far the most prevalent type of virus. NISTIR 7298 (Glossary of Key Information Security Terms, May 2013) defines a macro virus as a virus that attaches itself to documents and uses the macro programming capabilities of the document's application to execute and propagate. Macro viruses infect scripting code used to support active content in a variety of user document types. Macro viruses are particularly threatening for a number of reasons:

1. A macro virus is platform independent. Many macro viruses infect active content in commonly used applications, such as macros in Microsoft Word documents or other Microsoft Office documents, or scripting code in Adobe PDF documents. Any hardware platform and operating system that supports these applications can be infected.
2. Macro viruses infect documents, not executable portions of code. Most of the information introduced onto a computer system is in the form of documents rather than programs.
3. Macro viruses are easily spread, as the documents they exploit are shared in normal use. A very common method is by electronic mail, particularly since these documents can sometimes be opened automatically without prompting the user.
4. Because macro viruses infect user documents rather than system programs, traditional file system access controls are of limited use in preventing their spread, since users are expected to modify them.
5. Macro viruses are much easier to write or to modify than traditional executable viruses.

Macro viruses take advantage of support for active content using a scripting or macro language, embedded in a word processing document or other type of file. Typically, users employ macros to automate repetitive tasks and thereby save keystrokes. They are also used to support dynamic content, form validation, and other useful tasks associated with these documents.

Microsoft Word and Excel documents are common targets due to their widespread use. Successive releases of MS Office products provide increased protection against macro viruses. For example, Microsoft offers an optional Macro Virus Protection tool that detects suspicious Word files and alerts the customer to the potential risk of opening a file with macros. Office 2000 improved macro security by allowing macros to be digitally signed by their author, and for authors to be listed as trusted. Users were then warned if a document being opened contained unsigned, or signed but untrusted, macros, and were advised to disable macros in this case. Various antivirus product vendors have also developed tools to detect and remove macro viruses. As in other types of malware, the arms race continues in the field of macro viruses, but they no longer are the predominant malware threat.

Another possible host for macro virus-style malware is in Adobe's PDF documents. These can support a range of embedded components, including Javascript and other types of scripting code. Although recent PDF viewers include measures to warn users when such code is run, the message the user is shown can be manipulated to trick them into permitting its execution. If this occurs, the code could potentially act as a virus to infect other PDF documents the user can access on their system. Alternatively, it can install a Trojan, or act as a worm

Viruses Classification

There has been a continuous arms race between virus writers and writers of anti-virus software since viruses first appeared. As effective countermeasures are developed for existing types of viruses, newer types are developed.

A virus classification by target includes the following categories:

- **Boot sector infector:** Infects a master boot record or boot record and spreads when a system is booted from the disk containing the virus.
- **File infector:** Infects files that the operating system or shell consider to be executable.
- **Macro virus:** Infects files with macro or scripting code that is interpreted by an application.
- **Multipartite virus:** Infects files in multiple ways. Typically, the multipartite virus is capable of infecting multiple types of files, so virus eradication must deal with all of the possible sites of infection.

A virus classification by concealment strategy includes the following categories:

- **Encrypted virus:** A form of virus that uses encryption to obscure its content. A portion of the virus creates a random encryption key and encrypts the remainder of the virus. The key is stored with the virus. When an infected program is invoked, the virus uses the stored random key to decrypt the virus. When the virus replicates, a different random key is selected. Because the bulk of the virus is encrypted with a different key for each instance, there is no constant bit pattern to observe.
- **Stealth virus:** A form of virus explicitly designed to hide itself from detection by anti-virus software. Thus, the entire virus, not just a payload, is hidden. It may use code mutation, compression, or rootkit techniques to achieve this.
- **Polymorphic virus:** A form of virus that creates copies during replication that are functionally equivalent but have distinctly different bit patterns, in order to defeat programs that scan for viruses. In this case, the “signature” of the virus will vary with each copy. To achieve this variation, the virus may randomly insert superfluous instructions or interchange the order of independent instructions. A more effective approach is to use encryption. The strategy of the encryption virus is followed. The portion of the virus that is responsible for generating keys and performing encryption/decryption is referred to as the mutation engine. The mutation engine itself is altered with each use.
- **Metamorphic virus:** As with a polymorphic virus, a metamorphic virus mutates with every infection. The difference is that a metamorphic virus rewrites itself completely at each iteration, using multiple transformation techniques, increasing the difficulty of detection. Metamorphic viruses may change their behavior as well as their appearance.

6.3 Worm, Worm propagation Model, State of Worm Technology

A worm is a program that actively seeks out more machines to infect, and then each infected machine serves as an automated launching pad for attacks on other machines. Worm programs exploit software vulnerabilities in client or server programs to gain access to each new system. They can use network connections to spread from system to system. They can also spread through shared media, such as USB drives or CD and DVD data disks. E-mail worms can spread in macro or script code included in documents attached to e-mail or to instant messenger file transfers. Upon activation, the worm may replicate and propagate again. In addition to propagation, the worm usually carries some form of payload.

The concept of a computer worm was introduced in John Brunner's 1975 SF novel *The Shockwave Rider*. The first known worm implementation was done in Xerox Palo Alto Labs in the early 1980s. It was non-malicious, searching for idle systems to use to run a computationally intensive task.

To replicate itself, a worm uses some means to access remote systems. These include the following, most of which are still seen in active use:

- **Electronic mail or instant messenger facility:** A worm e-mails a copy of itself to other systems, or sends itself as an attachment via an instant message service, so that its code is run when the e-mail or attachment is received or viewed.
- **File sharing:** A worm either creates a copy of itself or infects other suitable files as a virus on removable media such as a USB drive; it then executes when the drive is connected to another system using the autorun mechanism by exploiting some software vulnerability, or when a user opens the infected file on the target system.
- **Remote execution capability:** A worm executes a copy of itself on another system, either by using an explicit remote execution facility or by exploiting a program flaw in a network service to subvert its operations.
- **Remote file access or transfer capability:** A worm uses a remote file access or transfer service to another system to copy itself from one system to the other, where users on that system may then execute it.
- **Remote login capability:** A worm logs onto a remote system as a user and then uses commands to copy itself from one system to the other, where it then executes.

The new copy of the worm program is then run on the remote system where, in addition to any payload functions that it performs on that system, it continues to propagate.

A worm typically uses the same phases as a computer virus: dormant, propagation, triggering, and execution. The propagation phase generally performs the following functions:

- Search for appropriate access mechanisms on other systems to infect by examining host tables, address books, buddy lists, trusted peers, and other similar repositories of remote system access details; by scanning possible target host addresses; or by searching for suitable removable media devices to use.
- Use the access mechanisms found to transfer a copy of itself to the remote system, and cause the copy to be run.

The worm may also attempt to determine whether a system has previously been infected before copying itself to the system. In a multiprogramming system, it can also disguise its presence by naming itself as a system process or using some other name that may not be noticed by a system operator. More recent worms can even inject their code into existing processes on the system, and run using additional threads in that process, to further disguise their presence.

Target Discovery

The first function in the propagation phase for a network worm is for it to search for other systems to infect, a process known as scanning or fingerprinting. For such worms, which exploit software vulnerabilities in remotely accessible network services, it must identify potential systems running the vulnerable service, and then infect them. Then, typically, the worm code now installed on the infected

machines repeats the same scanning process, until a large distributed network of infected machines is created.

Different types of network address scanning strategies that such a worm can use are:

- Random: Each compromised host probes random addresses in the IP address space, using a different seed. This technique produces a high volume of Internet traffic, which may cause generalized disruption even before the actual attack is launched.
- Hit-List: The attacker first compiles a long list of potential vulnerable machines. This can be a slow process done over a long period to avoid detection that an attack is underway. Once the list is compiled, the attacker begins infecting machines on the list. Each infected machine is provided with a portion of the list to scan. This strategy results in a very short scanning period, which may make it difficult to detect that infection is taking place.
- Topological: This method uses information contained on an infected victim machine to find more hosts to scan.
- Local subnet: If a host can be infected behind a firewall that host then looks for targets in its own local network. The host uses the subnet address structure to find other hosts that would otherwise be protected by the firewall.

Worm Propagation Model

A well-designed worm can spread rapidly and infect massive numbers of hosts. It is useful to have a general model for the rate of worm propagation. Computer viruses and worms exhibit similar self-replication and propagation behavior to biological viruses. Thus we can look to classic epidemic models for understanding computer virus and worm propagation behavior. A simplified, classic epidemic model can be expressed as follows:

$$dI(t)/dt = \beta I(t) S(t)$$

Where,

$I(t)$ = number of individuals infected as of time t .

$S(t)$ = number of susceptible individuals (susceptible to infection by not yet finished) at time t .

β = infection rate

N = size of the population, $N = I(t) + S(t)$

Figure 6.2 shows the dynamics of worm propagation using this model. Propagation proceeds through three phases. In the initial phase, the number of hosts increases exponentially. To see that this is so, consider a simplified case in which a worm is launched from a single host and infects two nearby hosts. Each of these hosts infects two more hosts, and so on. This results in exponential growth. After a time, infecting hosts waste some time attacking already infected hosts,

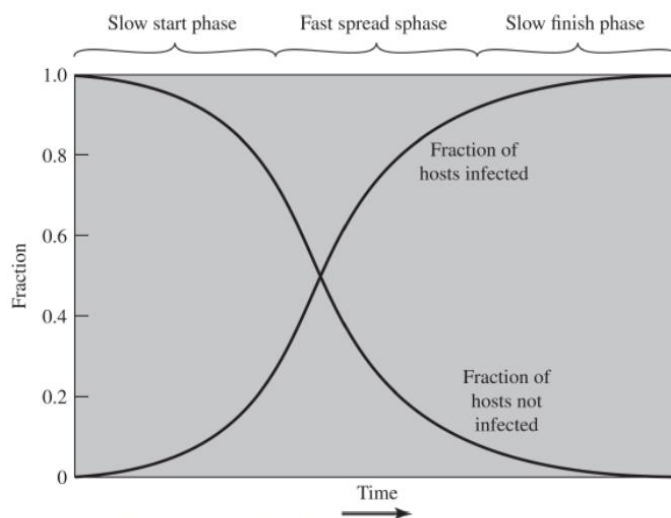


Figure 6.2 Worm Propagation Model

which reduces the rate of infection. During this middle phase, growth is approximately linear, but the rate of infection is rapid. When most vulnerable computers have been infected, the attack enters a slow finish phase as the worm seeks out those remaining hosts that are difficult to identify.

Clearly, the objective in countering a worm is to catch the worm in its slow start phase, at a time when few hosts have been infected.

State of Worm Technology

The state of the art in worm technology includes the following:

- **Multiplatform:** Newer worms are not limited to Windows machines but can attack a variety of platforms, especially the popular varieties of UNIX; or exploit macro or scripting languages supported in popular document types.
- **Multi-exploit:** New worms penetrate systems in a variety of ways, using exploits against Web servers, browsers, e-mail, file sharing, and other network-based applications; or via shared media.
- **Ultrafast spreading:** Exploit various techniques to optimize the rate of spread of a worm to maximize its likelihood of locating as many vulnerable machines as possible in a short time period.
- **Polymorphic:** To evade detection, skip past filters, and foil real-time analysis, worms adopt virus polymorphic techniques. Each copy of the worm has new code generated on the fly using functionally equivalent instructions and encryption techniques.
- **Metamorphic:** In addition to changing their appearance, metamorphic worms have a repertoire of behavior patterns that are unleashed at different stages of propagation.
- **Transport vehicles:** Because worms can rapidly compromise a large number of systems, they are ideal for spreading a wide variety of malicious payloads, such as distributed denial-of-service bots, rootkits, spam e-mail generators, and spyware.
- **Zero-day exploit:** To achieve maximum surprise and distribution, a worm should exploit an unknown vulnerability that is only discovered by the general network community when the worm is launched. In 2015, 54 zero-day exploits were discovered and exploited, significantly more than in previous years. Many of these were in common computer and mobile software. Some, though, were in common libraries and development packages, and some in industrial control systems. This indicates the range of systems being targeted.

6.4 Trojan Horse

A Trojan horse is a useful, or apparently useful, program or utility containing hidden code that, when invoked, performs some unwanted or harmful function.

Trojan horse programs can be used to accomplish functions indirectly that the attacker could not accomplish directly. For example, to gain access to sensitive, personal information stored in the files of a user, an attacker could create a Trojan horse program that, when executed, scans the user's files for the desired sensitive information and sends a copy of it to the attacker via a webform or e-mail or text message. The author could then entice users to run the program by incorporating it into a game or useful utility program, and making it available via a known software distribution site or app store. This approach has been used recently with utilities that "claim" to be the latest anti-virus scanner, or security update, for systems, but which are actually malicious Trojans, often carrying payloads such as spyware that

searches for banking credentials. Hence, users need to take precautions to validate the source of any software they install.

Trojan horses fit into one of three models:

- Continuing to perform the function of the original program and additionally performing a separate malicious activity.
- Continuing to perform the function of the original program but modifying the function to perform malicious activity (e.g., a Trojan horse version of a login program that collects passwords) or to disguise other malicious activity (e.g., a Trojan horse version of a process listing program that does not display certain processes that are malicious).
- Performing a malicious function that completely replaces the function of the original program.

Some Trojans avoid the requirement for user assistance by exploiting some software vulnerability to enable their automatic installation and execution. In this, they share some features of a worm, but unlike it, they do not replicate. A prominent example of such an attack was the Hydraq Trojan used in Operation Aurora in 2009 and early 2010. This exploited a vulnerability in Internet Explorer to install itself, and targeted several high-profile companies. It was typically distributed using either spam e-mail or via a compromised website using a “watering-hole” attack.

Tech Support Scams are a growing social engineering concern. These involve call centers calling users about non-existent problems on their computer systems. If the users respond, the attackers try to sell them bogus tech support or ask them to install Trojan malware or other unwanted applications on their systems, all while claiming this will fix their problem.

Mobile Phone Trojans

Mobile phone Trojans also first appeared in 2004 with the discovery of Skulker. As with mobile worms, the target is the smartphone, and the early mobile Trojans targeted Symbian phones. More recently, a significant number of Trojans have been detected that target Android phones and Apple iPhones. These Trojans are usually distributed via one or more of the app marketplaces for the target phone O/S.

The rapid growth in smartphone sales and use, which increasingly contain valuable personal information, make them an attractive target for criminals and other attackers. The number of vulnerabilities discovered in, and malware families targeting these phones, have both increased steadily in recent years. Recent examples include a phishing Trojan that tricks the user into entering their banking details, and ransomware that mimics Google’s design style to appear more legitimate and intimidating.

The tighter controls that Apple impose on their app store, mean that many iPhone Trojans target “jail-broken” phones, and are distributed via unofficial sites. However a number of versions of the iPhone O/S contained some form of graphic or PDF vulnerability. Indeed these vulnerabilities were the main means used to “jail-break” the phones. But they also provided a path that malware could use to target the phones. While Apple has fixed a number of these vulnerabilities, new variants continued to be discovered.

6.5 Intrusion and Intruders

One of the key threats to security is the use of some form of hacking by an intruder, often referred to as a hacker or cracker. Verizon indicates that 92% of the breaches they investigated were by outsiders, with 14% by insiders, and with some breaches involving both outsiders and insiders. They also noted that insiders were responsible for a small number of very large dataset compromises. Both Symantec and

Verizon also comment that not only is there a general increase in malicious hacking activity, but also an increase in attacks specifically targeted at individuals in organizations and the IT systems they use. This trend emphasizes the need to use defense-in-depth strategies, since such targeted attacks may be designed to bypass perimeter defenses such as firewalls and network-based Intrusion detection systems (IDSs).

Different classes of intruders are:

- **Cyber criminals:** Are either individuals or members of an organized crime group with a goal of financial reward. To achieve this, their activities may include identity theft, theft of financial credentials, corporate espionage, data theft, or data ransomware. They often meet in underground forums with names such as DarkMarket.org and theftservices.com to trade tips and data and coordinate attacks.
- **Activists:** Are either individuals working as insiders, or members of a larger group of outsider attackers, who are motivated by social or political causes. They are also known as hacktivists, and their skill level may be quite low. The aim of their attacks is often to promote and publicize their cause, typically through website defacement, denial of service attacks, or the theft and distribution of data that results in negative publicity or compromise of their targets.
- **State-sponsored organizations:** Are groups of hackers sponsored by governments to conduct espionage or sabotage activities. They are also known as Advanced Persistent Threats (APTs), due to the covert nature and persistence over extended periods involved with many attacks in this class.
- **Others:** Are hackers with motivations other than those listed above, including classic hackers or crackers who are motivated by technical challenge or by peer-group esteem and reputation. Many of those responsible for discovering new categories of buffer overflow vulnerabilities

Across these classes of intruders, there is also a range of skill levels seen. These can be broadly classified as:

- **Apprentice:** Hackers with minimal technical skill who primarily use existing attack toolkits. They likely comprise the largest number of attackers, including many criminal and activist attackers. Given their use of existing known tools, these attackers are the easiest to defend against. They are also known as “script-kiddies” due to their use of existing scripts (tools).
- **Journeyman:** Hackers with sufficient technical skills to modify and extend attack toolkits to use newly discovered, or purchased, vulnerabilities; or to focus on different target groups. They may also be able to locate new vulnerabilities to exploit that are similar to some already known. A number of hackers with such skills are likely found in all intruder classes listed above, adapting tools for use by others. The changes in attack tools make identifying and defending against such attacks harder.
- **Master:** Hackers with high-level technical skills capable of discovering brand new categories of vulnerabilities, or writing new powerful attack toolkits. Some of the better-known classical hackers are of this level, as clearly are some of those employed by some state-sponsored organizations, as the designation APT suggests. This makes defending against these attackers of the highest difficulty.

Intruder attacks range from the benign to the serious. At the benign end of the scale, there are people who simply wish to explore the Internet and see what is out there. At the serious end are individuals

or groups that attempt to read privileged data, perform unauthorized modifications to data, or disrupt systems.

NIST SP 800-61 (Computer Security Incident Handling Guide, August 2012) lists the following examples of intrusion:

- Performing a remote root compromise of an e-mail server
- Defacing a Web server
- Guessing and cracking passwords
- Copying a database containing credit card numbers
- Viewing sensitive data, including payroll records and medical information, without authorization
- Running a packet sniffer on a workstation to capture usernames and passwords
- Using a permission error on an anonymous FTP server to distribute pirated software and music files
- Dialing into an unsecured modem and gaining internal network access
- Posing as an executive, calling the help desk, resetting the executive's e-mail password, and learning the new password
- Using an unattended, logged-in workstation without permission

Intruder Behavior

The techniques and behavior patterns of intruders are constantly shifting to exploit newly discovered weaknesses and to evade detection and countermeasures. However, intruders typically use steps from a common attack methodology such as:

- **Target Acquisition and Information Gathering:** Where the attacker identifies and characterizes the target systems using publicly available information, both technical and non-technical, and the use of network exploration tools to map target resources.
- **Initial Access:** The initial access to a target system, typically by exploiting a remote network vulnerability, by guessing weak authentication credentials used in a remote service, or via the installation of malware on the system using some form of social engineering or drive-by-download attack.
- **Privilege Escalation:** Actions taken on the system, typically via a local access vulnerability, to increase the privileges available to the attacker to enable their desired goals on the target system.
- **Information Gathering or System Exploit:** Actions by the attacker to access or modify information or resources on the system, or to navigate to another target system.
- **Maintaining Access:** Actions such as the installation of backdoors or other malicious software, or through the addition of covert authentication credentials or other configuration changes to the system, to enable continued access by the attacker after the initial attack.
- **Covering Tracks:** Where the attacker disables or edits audit logs, to remove evidence of attack activity, and uses rootkits and other measures to hide covertly installed files or code.

6.6 Intrusion Detection System

Intrusion detection system is a hardware or software component that gathers and analyzes information from various areas within a computer or a network to identify possible security intrusions.

An IDS comprises three logical components:

- **Sensors:** Sensors are responsible for collecting data. The input for a sensor may be any part of a system that could contain evidence of an intrusion. Types of input to a sensor includes network packets, log files, and system call traces. Sensors collect and forward this information to the analyzer.
- **Analyzers:** Analyzers receive input from one or more sensors or from other analyzers. The analyzer is responsible for determining if an intrusion has occurred. The output of this component is an indication that an intrusion has occurred. The output may include evidence supporting the conclusion that an intrusion occurred. The analyzer may provide guidance about what actions to take as a result of the intrusion. The sensor inputs may also be stored for future analysis and review in a storage or database component.
- **User interface:** The user interface to an IDS enables a user to view output from the system or control the behavior of the system. In some system, the user interface may equate to a manager, director, or console component.

An IDS may use a single sensor and analyzer, such as a classic HIDS on a host or NIDS in a firewall device. More sophisticated IDSs can use multiple sensors, across a range of host and network devices, sending information to a centralized analyzer and user interface in a distributed architecture.

IDSs are often classified based on the source and type of data analyzed, as:

- **Host-based IDS (HIDS):** Monitors the characteristics of a single host and the events occurring within that host, such as process identifiers and the system calls they make, for evidence of suspicious activity.
- **Network-based IDS (NIDS):** Monitors network traffic for particular network segments or devices and analyzes network, transport, and application protocols to identify suspicious activity.
- **Distributed or hybrid IDS:** Combines information from a number of sensors, often both host and network-based, in a central analyzer that is able to better identify and respond to intrusion activity.

Basic Principles

Authentication facilities, access control facilities, and firewalls all play a role in countering intrusions. Another line of defense is intrusion detection, and this has been the focus of much research in recent years. This interest is motivated by a number of considerations, including the following:

1. If an intrusion is detected quickly enough, the intruder can be identified and ejected from the system before any damage is done or any data are compromised. Even if the detection is not sufficiently timely to preempt the intruder, the sooner that the intrusion is detected, the less the amount of damage and the more quickly that recovery can be achieved.
2. An effective IDS can serve as a preventive, thus acting to prevent intrusions.
3. Intrusion detection enables the collection of information about intrusion techniques that can be used to strengthen intrusion prevention measures.

Intrusion detection is based on the assumption that the behavior of the intruder differs from that of a legitimate user in ways that can be quantified. Of course, we cannot expect that there will be a crisp, exact distinction between an attack by an intruder and the normal use of resources by an authorized user. Rather, we must expect that there will be some overlap.

6.7 Analysis Approaches: Anomaly Based, Signature Based

IDSs typically use one of the following alternative approaches to analyze sensor data to detect intrusions:

1. **Anomaly detection:** Involves the collection of data relating to the behavior of legitimate users over a period of time. Then, current observed behavior is analyzed to determine with a high level of confidence whether this behavior is that of a legitimate user or alternatively that of an intruder.
2. **Signature or Heuristic detection:** Uses a set of known malicious data patterns (signatures) or attack rules (heuristics) that are compared with current behavior to decide if it is that of an intruder. It is also known as misuse detection. This approach can only identify known attacks for which it has patterns or rules.

In essence, anomaly approaches aim to define normal, or expected, behavior, in order to identify malicious or unauthorized behavior. Signature or heuristic-based approaches directly define malicious or unauthorized behavior. They can quickly and efficiently identify known attacks. However, only anomaly detection is able to detect unknown, zero-day attacks, as it starts with known good behavior and identifies anomalies to it. Given this advantage, clearly anomaly detection would be the preferred approach, were it not for the difficulty in collecting and analyzing the data required, and the high level of false alarms.

Anomaly Detection

The anomaly detection approach involves first developing a model of legitimate user behavior by collecting and processing sensor data from the normal operation of the monitored system in a training phase. This may occur at distinct times, or there may be a continuous process of monitoring and evolving the model over time. Once this model exists, current observed behavior is compared with the model in order to classify it as either legitimate or anomalous activity in a detection phase. A variety of classification approaches are used, such as:

- **Statistical:** Analysis of the observed behavior using univariate, multivariate, or time-series models of observed metrics.
- **Knowledge based:** Approaches use an expert system that classifies observed behavior according to a set of rules that model legitimate behavior.
- **Machine-learning:** Approaches automatically determine a suitable classification model from the training data using data mining techniques.

A key limitation of anomaly detection approaches used by IDSs, particularly the machine-learning approaches, is that they are generally only trained with legitimate data, unlike many of the other applications where both legitimate and anomalous training data is used. The lack of anomalous training data, which occurs given the desire to detect currently unknown future attacks, limits the effectiveness.

Signature or Heuristic Detection

Signature or heuristic techniques detect intrusion by observing events in the system and applying either a set of signature patterns to the data, or a set of rules that characterize the data, leading to a decision regarding whether the observed data indicates normal or anomalous behavior.

Signature approaches match a large collection of known patterns of malicious data against data stored on a system or in transit over a network. The signatures need to be large enough to minimize the false alarm rate, while still detecting a sufficiently large fraction of malicious data. This approach is widely used in antivirus products, in network traffic scanning proxies, and in NIDS.

The advantages of this approach include the relatively low cost in time and resource use, and its wide acceptance.

Disadvantages include the significant effort required to constantly identify and review new malware to create signatures able to identify it, and the inability to detect zero-day attacks for which no signatures exist.

Rule-based heuristic identification involves the use of rules for identifying known penetrations or penetrations that would exploit known weaknesses. Rules can also be defined that identify suspicious behavior, even when the behavior is within the bounds of established patterns of usage. Typically, the rules used in these systems are specific to the machine and operating system. The most fruitful approach to developing such rules is to analyze attack tools and scripts collected on the Internet. These rules can be supplemented with rules generated by knowledgeable security personnel. In this latter case, the normal procedure is to interview system administrators and security analysts to collect a suite of known penetration scenarios and key events that threaten the security of the target system.

6.8 Honeypots

Honeypots are trap systems that are designed to lure a potential attacker away from critical systems. Honeypots are designed to:

- Divert an attacker from accessing critical systems.
- Collect information about the attacker's activity.
- Encourage the attacker to stay on the system long enough for administrators to respond.

These systems are filled with fabricated information designed to appear valuable but that a legitimate user of the system would not access. Thus, any access to the honeypot is suspect. The system is instrumented with sensitive monitors and event loggers that detect these accesses and collect information about the attacker's activities. Because any attack against the honeypot is made to seem successful, administrators have time to mobilize and log and track the attacker without ever exposing productive systems.

The honeypot is a resource that has no production value. There is no legitimate reason for anyone outside the network to interact with a honeypot. Thus, any attempt to communicate with the system is most likely a probe, scan, or attack. Conversely, if a honeypot initiates outbound communication, the system has probably been compromised.

Honeypots are typically classified as being either low or high interaction.

- Low interaction honeypot: Consists of a software package that emulates particular IT services or systems well enough to provide a realistic initial interaction, but does not execute a full version of those services or systems.
- High interaction honeypot: Is a real system, with a full operating system, services and applications, which are instrumented and deployed where they can be accessed by attackers.

A high interaction honeypot is a more realistic target that may occupy an attacker for an extended period. However, it requires significantly more resources, and if compromised could be used to initiate attacks on other systems. This may result in unwanted legal or reputational issues for the organization running it. A low interaction honeypot provides a less realistic target, able to identify intruders using the earlier stages

of the attack methodology. This is often sufficient for use as a component of a distributed IDS to warn of imminent attack. “The Honeynet Project” provides a range of resources and packages for such systems.

Honeypots can be deployed in a variety of locations. Figure 8.8 illustrates some possibilities. The location depends on a number of factors, such as the type of information the organization is interested in gathering and the level of risk that organizations can tolerate to obtain the maximum amount of data.

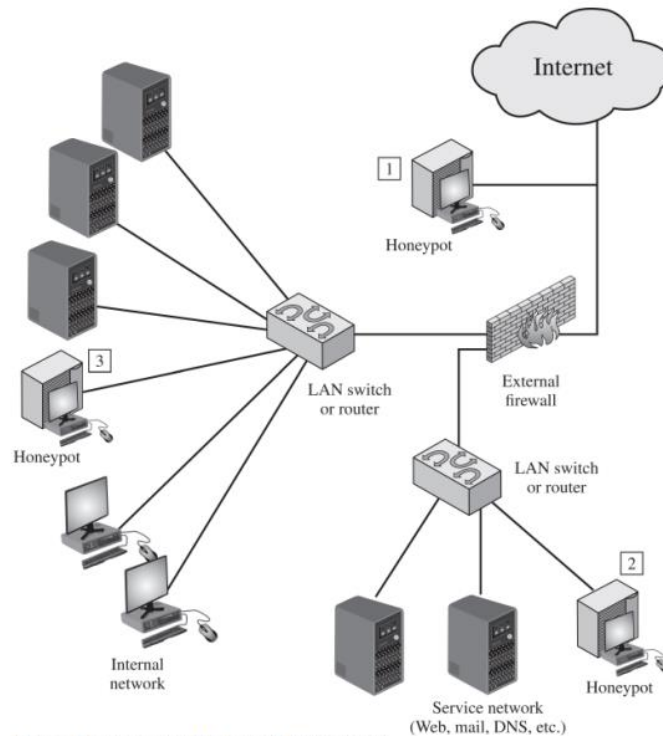


Figure 8.8 Example of Honeypot Deployment

A honeypot outside the external firewall (location 1) is useful for tracking attempts to connect to unused IP addresses within the scope of the network. A honeypot at this location does not increase the risk for the internal network. The danger of having a compromised system behind the firewall is avoided. Further, because the honeypot attracts many potential attacks, it reduces the alerts issued by the firewall and by internal IDS sensors, easing the management burden. The disadvantage of an external honeypot is that it has little or no ability to trap internal attackers, especially if the external firewall filters traffic in both directions.

The network of externally available services, such as Web and mail, often called the DMZ (demilitarized zone), is another candidate for locating a honeypot (location 2). The security administrator must assure that the other systems in the DMZ are secure against any activity generated by the honeypot. A disadvantage of this location is that a typical DMZ is not fully accessible, and the firewall typically blocks traffic to the DMZ the attempts to access unneeded services. Thus, the firewall either has to open up the traffic beyond what is permissible, which is risky, or limit the effectiveness of the honeypot.

A fully internal honeypot (location 3) has several advantages. Its most important advantage is that it can catch internal attacks. A honeypot at this location can also detect a misconfigured firewall that forwards impermissible traffic from the Internet to the internal network. There are several disadvantages. The most serious of these is if the honeypot is compromised so it can attack other internal systems. Any further

traffic from the Internet to the attacker is not blocked by the firewall because it is regarded as traffic to the honeypot only. Another difficulty for this honeypot location is that, as with location 2, the firewall must adjust its filtering to allow traffic to the honeypot, thus complicating firewall configuration and potentially compromising the internal network.

Practice Questions

1. What is a malicious software? Explain different types of malicious software in detail.
2. What are the characteristics of an advanced persistent threat?
3. What are typical phases of operation of a virus or worm? Explain.
4. Explain about virus and its types.
5. What means can a worm use to access remote systems to propagate?
6. How does a Trojan enable malware to propagate? How common are Trojans on computer systems? Or on mobile platforms?
7. List and briefly define classes of intruders.
8. List and briefly describe the steps typically used by intruders when attacking a system.
9. Describe the three logical components of an IDS.
10. Describe the differences between a host-based IDS and a network-based IDS. How can their advantages be combined into a single system?
11. What are the benefits that can be provided by an IDS?
12. What is the difference between anomaly detection and signature or heuristic intrusion detection?
13. What is a honeypot? Explain in detail.
14. List and briefly define the types of honeypots that may be deployed.